COMP0248: Objection Detection and Classification

Coursework 1 Description

Sophia Bano

MSc Robotics and AI
UCL Computer Science

Coursework 1: Hand Gesture Detection, Segmentation & Classification from RGB-D

- **Weight:** 40%

- Data collection & annotation - Submission: **12th February 2026**
- CW Submission deadline: **6th March 2026**

- **Dataset:** COMP0248 Hand Gesture RGB-D - collected by you during lab session 4 (4th Feb) and session 5 (11th Feb – only if needed)

- **Classes:** 10 predefined hand gestures (call, dislike, like, ok, one, palm, peace, rock, stop, three)

Coursework tasks

Objective 1: Collect 10 gestures dataset using Intel Realsense RGB-D camera and annotate. Follow the **Data Curation tutorial** provided in Lab 4.

Objective 2: Develop a deep learning algorithm that takes an image (RGB or RGB-D) of a hand performing one of 10 predefined gestures and

- **Detects** the hand (bounding box),
- **Segments** the hand (pixel-wise mask), and
- **Classifies** the gesture (10 classes)

Evaluation & Analysis:

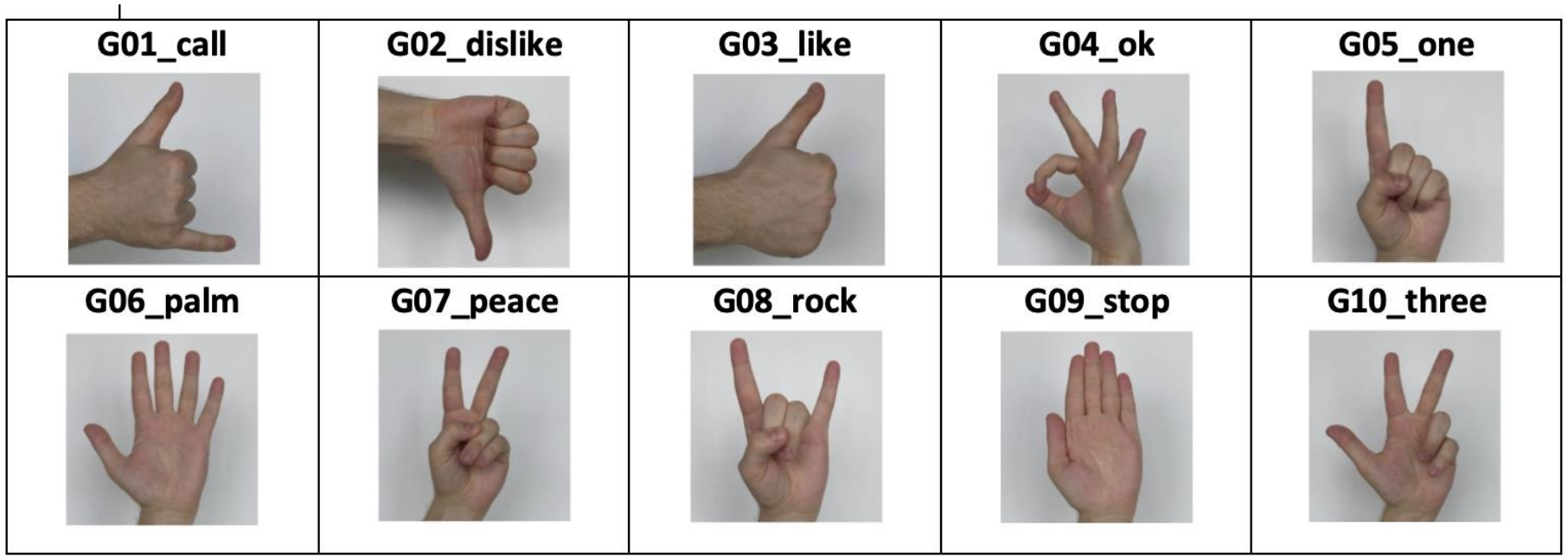
- Compare baseline vs your model: analyse errors and robustness.

Why Hand Gestures?

- **Human–robot interaction:** simple, intuitive command interface
- **Robotics & AR:** hands often occlude tools/objects - robust segmentation needed
- **Real-world variability:** background clutter, lighting change, different skin tones
- **RGB-D advantage:** depth simplifies segmentation and enables faster, more robust models

Our Dataset: COMP0248 Hand Gesture RGB-D

- **Training data:** 35 student contributors
- **Test data:** 5 additional holdout test subjects (to be released on 27th Feb)



Data Capturing and Annotation Constraints (1/3)

Each student **must capture and contribute** the following data:

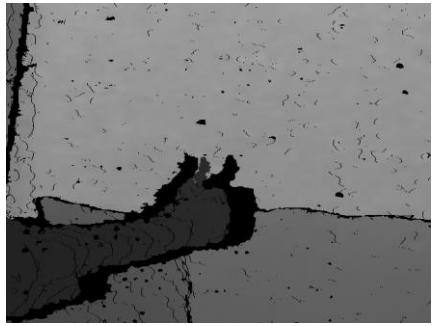
- 10 predefined gestures × 5 clips/gesture = **50 clips**
- **Clip length:** 5 seconds (consider changing hand pose while keeping the gesture fixed during data recording)
- **Data capture modality:** RGB Depth (Real sense D455) – cameras only provided during the lab session
- **Annotation masks:** each student must provide (single channel) hand masks for 2 keyframes per clip
  - Keyframes must be at least 1 sec apart and show complete hand in view.
  - Total annotations: 2 x 50 clips = 100 annotated frames per student

Data Capturing Constraints (2/3)

Use the provided script for data collection

- Camera: Intel RealSense RGB-D (D455)
- Resolution: 640 × 480 (fixed for all students)
- FPS: 3

RGB



Depth

- Capture data using only right hand (other hand must be out of frame)
- Face out of frame (shoulders allowed)
- Hand distance: roughly 60 – 100 cm from D455 (one arm left) – ensure depth map looks reliable.
- Hand should not be truncated; keep entire hand visible and possibly centred
- Select five different backgrounds for the 5 set of recordings.
  - E.g., uncluttered or cluttered background, varying illumination (in dark or bright room) – justify your background choice

Data Capturing Constraints (3/3)

For each student, the collected and annotated data must be stored in the following folder structure, "<studentno>_<surname>" zipped and uploaded on the provided Moodle link – **Deadline 12th Feb**

```
dataset/
└── <studentno>_<surname>/
    ├── G01_call/
    │   ├── clip01/
    │   │   ├── rgb/
    │   │   │   ├── frame_001.png
    │   │   │   ├── frame_002.png
    │   │   │   └── ...
    │   │   ├── depth/
    │   │   │   ├── frame_001.png
    │   │   │   ├── frame_002.png
    │   │   │   └── ...
    │   │   ├── depth_raw/
    │   │   │   ├── frame_001.npy
    │   │   │   ├── frame_002.npy
    │   │   │   └── ...
    │   │   └── annotation/
    │   │       ├── frame_00X.png   # only keyframes have masks (00X to be the same number as in the rgb folder)
    │   │       └── frame_00Y.png
    │   ├── clip02/
    │   │   ├── rgb/
    │   │   ├── depth/
    │   │   ├── depth_raw/
    │   │   └── annotation/
    │   ├── clip03/
    │   ├── clip04/
    │   └── clip05/
    ├── G02_dislike/
    │   ├── clip01/
    │   ├── clip02/
    │   ├── clip03/
    │   ├── clip04/
    │   └── clip05/
    ├── G03_like/
    │   ├── clip01/
    │   ├── clip02/
    │   ├── clip03/
    │   ├── clip04/
    │   └── clip05/
    ├── G04_ok/
    │   ├── clip01/
    │   └── ... (clip02–clip05)
    ├── G05_one/
    │   └── ... (clip01–clip05)
    ├── G06_palm/
    │   └── ... (clip01–clip05)
    ├── G07_peace/
    │   └── ... (clip01–clip05)
    ├── G08_rock/
    │   └── ... (clip01–clip05)
    ├── G09_stop/
    │   └── ... (clip01–clip05)
    └── G10_three/
        └── ... (clip01–clip05)
```

Collated training data (from all students) will be **released on 13th Feb**

You are free to define the **train-val split.**

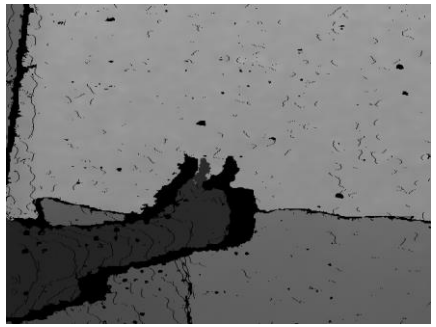
Note: Students to provide the data in the correct format. We take no responsibility in any format/folder structure mismatch issues.

Data Annotation

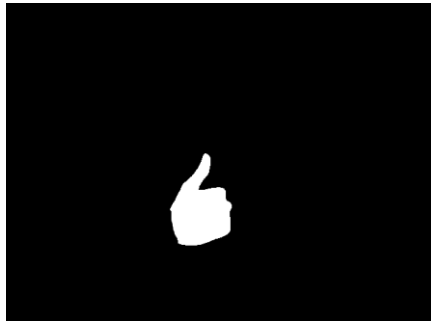
- Recommended annotation software and process is detailed in **Lab 4 tutorial**
  - Pre-annotating with SAM3 and refining in Label Studio

- Annotation must be stored with the same file name as the key frame as
  - single channel png files
  - 0 = background
  - 255 = hand
  - segment hand up to the wrist (consistent across dataset)

RGB



Depth



Annotated groundtruth (GT)

Evaluation & Analysis

Report at least the following metrics on val and test data (comparing also with a baseline)

- **Detection metrics:**
  – Detection accuracy@0.5 IoU
  – Mean bounding-box IoU across the val/test dataset

- **Segmentation metrics:**
  – Mean IoU for hand vs background
  – Dice coefficient
  – Qualitative mask (and detected box) overlays on a a few set of val/test images

- **Classification metrics:**
  – Overall top-1 accuracy
  – Macro-averaged F1 score across 10 gesture classes
  – Confusion matrix

Technical requirements (1/2)

Implement your own model(s) and training code in PyTorch.

Write your own:

- **custom model architecture** (or architectures) written using torch.nn.Module.
- **training loop**
- **loss computation** (compose built-in PyTorch losses in your own training loop)
- **evaluation scripts** for the evaluation metrics

Technical requirements (2/2)

High-Level Frameworks are not allowed

Following are **not allowed** as the main solution:

- High-level detection/segmentation frameworks that provide end-to-end models, including (but not limited to) Ultralytics YOLO, Detectron2, MMDetection, segmentation_models_pytorch, any pre-built Mask R-CNN / Faster R-CNN that you only fine-tune.
- Using torchvision.models.detection.maskrcnn_resnet50_fpn (or similar detection models) as your submitted model.

IMPORTANT: If you are unsure whether a particular library is allowed, assume it is not and ask for clarification.

Deliverables (1/2)

Submit

Project_studentname.zip

(with dataset folder only containing your collected folder)

Include a github link to your code (containing readme as well) in the report.

Code **MUST** be modular, documented, and reproducible.

```
project_<studentno>_<surname>/
├── dataset/
│   └── <studentno>_<surname>/      # your own recorded & annotated data only
│
├── src/
│   ├── dataloader.py               # data loading + preprocessing
│   ├── model.py                    # detection + segmentation + classification model(s)
│   ├── train.py                    # training loop(s)
│   ├── evaluate.py                 # all required metrics (det/seg/cls)
│   ├── visualise.py                # (optional) plotting masks, boxes, confusion matrices, etc.
│   └── utils.py                    # helper functions (metrics, visualisation helpers, etc.)
│
├── weights/                        # saved checkpoints (.pt/.pth)
├── results/                        # predictions, logs, plots, confusion matrices, etc.
├── requirements.txt                # Python dependencies / environment
└── README.md                       # setup + how to run training / evaluation
```

Deliverables (2/2)

Submit: coursework1_studentname.pdf

Technical Report (6-page LaTeX – in IEEE Conference Template) – including references

Indicative structure:

- **Introduction:** Problem statement, relevance to robotics.
- **Dataset:** 10 class distribution, selected scenarios, challenges, etc.
- **Methodology:** Model architecture, training details, design choices, justifications
- **Experimental setup:** hyperparameters, etc.
- **Results:** quantitative and qualitative results on val and test data
- **Innovation:** challenges handled, innovation introduced, comparison with baseline/youer earlier implementations
- **Discussion:** discuss results and limitations
- **Conclusion:** Lessons learned, future work.

Plagiarism

Group discussions are encouraged but the work must be student's original work.

Report writing:

- Must not contain AI-generated text (e.g., ChatGPT) or copied version.

Prohibited:

- Copying code/text without attribution.
- Using generative AI for report writing.

Penalty: 0% + misconduct referral

FAQs

Q: Does the 6-page limit include references and appendices?
A: Yes.

Q: What font size should be used for text in the report?
A: Use the provided template without making any alterations.

Q: Can I use GenAI tools (e.g., ChatGPT) to correct grammar and spelling errors?
A: No. The work must reflect your original writing style and present your own ideas and understanding. Spelling or grammar errors will not penalize your submission.

Q: Can I train models on my personal computer?
A: Your code must be compatible with the CS provided GPU servers. Hence, using the provided GPUs is strongly recommended.

Marking breakdown

Report: 40 marks

Code: 40 marks

Innovation in model: 20 marks

Total: 100 marks